

MODULE 3 | SESSION 6

Routing + Navigation

React Router | Nested Routes | URL Params | Zustand Auth | GT3 Part 1

ITELECT4 | AY 2026-2027 | Saturday, August 8, 2026

Session 6: Routing + Navigation

One app, five pages -- and a URL that finally means something

Topics

01	Client-Side Routing	what a router actually replaces
02	Pages + Route Table	splitting App.tsx into src/pages/
03	Layout + Outlet	one nav bar, shared by every page
04	URL Parameters	typed useParams and useNavigate
05	Auth + ProtectedRoute	a Zustand token gates a route

GT3 - Part 1 of 3

Graded Task -- push today, no tag

- Add React Router v8 to your own app -- at least 4 routes
- One Layout component with a shared nav and an Outlet
- One route with a URL parameter, read with a typed useParams
- A typed Zustand auth store -- token, login, logout
- A ProtectedRoute that redirects to /login when there is no token
- Work on a branch named gt3-part1 and open a pull request

Quick Recap: Session 5

Where GT2 left the app -- and the one thing it still cannot do

- **src/App.tsx** holds *everything*: the mock data, all nine pieces of state, the search handler, and every line of JSX
- **UserCard**, **CourseCard** and **SubmissionBadge** are styled with Tailwind utility classes and **dark**: variants
- **useToggle** powers the dark mode toggle; **usePrevious** tracks the previous search term
- Tailwind is installed and working -- the **@import** line plus **@custom-variant dark** in **src/index.css**
- The app has exactly *one* screen. There is no link, no menu, and no way to get anywhere else.

Today we split that one screen into five pages -- and give each page its own URL.

The Problem: One Page, One File

Why we add a router before we add anything else to this app

- **A real app has more than one screen.** A dashboard, a course list, a detail view, a login page. Right now our entire app is one *return* statement.
- **Users expect the URL to mean something.** They bookmark pages, press Back, and paste links to each other. A single-URL app can do *none* of that.
- **The obvious fix makes it worse.** You could add a **page** state and a giant ternary -- but Back still would not work, and **App.tsx** would only get longer.
- **A router is the real fix.** It maps each URL to exactly one component, and keeps the address bar, the Back button and the browser history all working for free.

```
// The naive way -- do NOT do this
const [page, setPage] = useState("home");

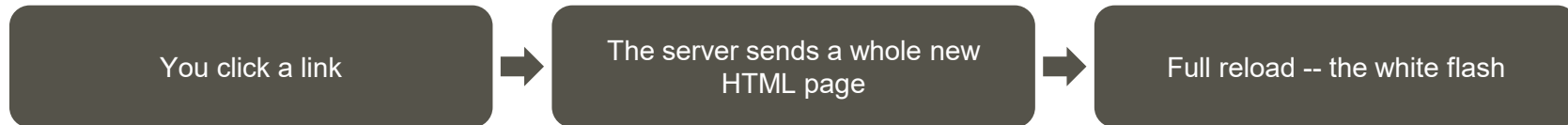
return (
  <div>
    {page === "home" && <Dashboard />}
    {page === "courses" && <Courses />}
  </div>
);

// The URL never changes.
// Back leaves your app entirely.
// Nothing can be bookmarked.
```

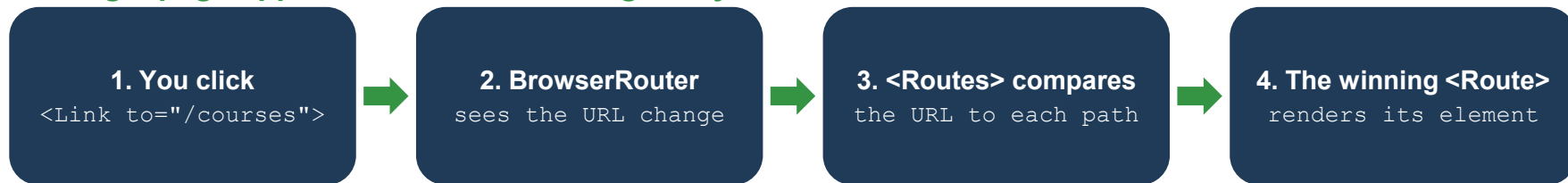
How Client-Side Routing Works

The router never asks the server for a new page -- it swaps a component

A traditional website -- every click is a new page from the server



A single-page app -- what we are building today



No request to the server. No page reload.

The address bar changes, the Back button works, and only the component inside <Outlet /> is replaced.

Important: v6, v7, and v8

Almost every tutorial you will find online is out of date. Read this before you install.

- **The package react-router-dom no longer exists.** It was removed in **v8** (June 2026). It had only ever been a mirror of **react-router**.
- **We install react-router.** Today **npm install react-router** gives you **v8.3.0**. Everything we use imports from that one package.
- **The API is the same as v6.** **Routes, Route, Link, Outlet, useParams, useNavigate** -- same names, same behaviour. *Only the import line changed.*
- **So when a tutorial says react-router-dom,** change that one word to **react-router** and the rest of the code works unchanged.

```
// WRONG -- the old package (v6 / v7)
import { Routes } from "react-router-dom";
// Cannot find module
//   'react-router-dom'

// RIGHT -- what we install today (v8)
import { Routes } from "react-router";

// The one exception, which we do
// not use in this course:
// RouterProvider and HydratedRouter
// live in "react-router-dom".
// Everything else: "react-router".
```

Installing React Router

In the following slides, we will add:

- **react-router** (v8) installed with npm, together with **zustand**, which the last section needs
- **BrowserRouter** wrapped around **<App />** inside **src/main.tsx**
- Why the router has to sit *above* every component that wants to navigate
- Nothing visible changes yet -- this step only switches the router on

File: itelect4-project/ (terminal), then **src/main.tsx**

Installing React Router

Two steps: the terminal first, then one file.

```
// ----- STEP 1: terminal, inside itelect4-project -----
> npm install react-router zustand

// ----- STEP 2: src/main.tsx (REPLACE the whole file) -----
import { StrictMode } from "react";
import { createRoot } from "react-dom/client";
import { BrowserRouter } from "react-router";      // <-- NEW
import "./index.css";
import App from "./App.tsx";

createRoot(document.getElementById("root")!).render(
  <StrictMode>
    <BrowserRouter>                { /* <-- NEW */ }
    <App />
  </BrowserRouter>                { /* <-- NEW */ }
</StrictMode>,
);

// You should now see: nothing different at all. The app looks exactly
// like it did last week. That is correct -- the router is switched on,
// but we have not given it a single route to match yet.
```


Installing React Router

Code Explanation

- **npm install react-router zustand** -- one command, two packages. **zustand** is not used until the last section today, but installing it now saves a mid-lab interruption.
- **<BrowserRouter>** -- This tells React: "watch the browser's address bar, and tell everything inside me the moment the URL changes."
- **It wraps <App />, not the other way round.** Any component that calls **useNavigate** or renders a **<Link>** must be *inside* this wrapper, or it throws at runtime.
- **{/* ... */}** is how you comment inside JSX. A plain *//* in that position is *not* a comment -- React prints it on the page as ordinary text.

```
// src/main.tsx
<StrictMode>
  <BrowserRouter>
    <App />
  </BrowserRouter>
</StrictMode>

// Inside JSX:
{/* this is a comment */}
// this one gets PRINTED
```

Splitting App.tsx Into Pages

In the following slides, we will add:

- A new **src/data/mockData.ts** so more than one page can share the same **student** and **courses**
- A new **src/pages/** folder -- one file per screen, starting with **DashboardPage.tsx**
- **src/App.tsx** stops rendering any UI at all and becomes a *route table* instead
- Nothing from Session 5 is thrown away -- the code *moves*, almost unchanged

Files: **src/data/mockData.ts**, **src/pages/DashboardPage.tsx**, **src/App.tsx**

Shared Mock Data

A NEW file: src/data/mockData.ts. Create the src/data/ folder first.

```
// src/data/mockData.ts -- NEW FILE
// Session 5 kept `student` and `course` at the top of App.tsx. Several
// pages need that data now, so it moves into its own file.
import type { User, Course, Submission } from "../types/index";

export const student: User = {
  id: 1, name: "Juan dela Cruz", email: "juan@example.com",
  role: "student", isActive: true,
};

export const allCourses: Course[] = [
  { code: "ITELECT4", title: "IT Elective 4",
    units: 3, semester: "1st Semester 2026-2027" },
  { code: "ITELECT3", title: "IT Elective 3",
    units: 3, semester: "2nd Semester 2025-2026" },
  { code: "CSSWENG", title: "Software Engineering",
    units: 3, semester: "1st Semester 2026-2027" },
];

export const allSubmissions: Submission[] = [
  { id: 1, studentId: 1, courseCode: "ITELECT4",
    repoUrl: "github.com/juan/itelect4-project",
    submittedAt: new Date(), score: 95 },
  { id: 2, studentId: 1, courseCode: "ITELECT3",
    repoUrl: "github.com/juan/itelect3-final",
    submittedAt: new Date() },
];
```

The First Page (1 of 2)

A NEW file: `src/pages/DashboardPage.tsx`. This is Session 5's `App.tsx` body, moved.

```
// src/pages/DashboardPage.tsx -- NEW FILE
// Every import path gains a ../ because this file now sits one
// folder deeper, inside src/pages/ instead of src/.
import { useState } from "react";
import type { User } from "../types/index";
import UserCard from "../components/UserCard";
import useToggle from "../hooks/useToggle";
import { student } from "../data/mockData";

function DashboardPage() {
  // These two came straight from Session 5's App.tsx, unchanged
  const [selectedUser, setSelectedUser] = useState<User | null>(null);
  const [showDetails, toggleDetails] = useToggle(false);

  return (
    <div>
      <h2 className="mb-4 text-2xl font-bold text-gray-900
        dark:text-white">
        Dashboard
      </h2>

      // ... the JSX continues on the next slide -- do not type this line ...
```

The First Page (2 of 2)



src/pages/DashboardPage.tsx, continued -- this closes the file.

```
// src/pages/DashboardPage.tsx -- continues directly from the previous
// slide, still inside the same return ( ... )

<div className="grid grid-cols-1 gap-4 sm:grid-cols-2
  lg:grid-cols-3">
  <UserCard user={student} onSelect={setSelectedUser} />
</div>

<button onClick={toggleDetails}
  className="mt-4 rounded bg-gray-200 px-3 py-1.5 text-sm
  dark:bg-gray-700 dark:text-white">
  {showDetails ? "Hide" : "Show"} Details
</button>

{showDetails && selectedUser !== null && (
  <p className="mt-2 text-gray-700 dark:text-gray-300">
    Selected: {selectedUser.name} ({selectedUser.role})
  </p>
)}
</div>
);
}

export default DashboardPage;
```

App.tsx Becomes a Route Table

In src/App.tsx: DELETE everything, REPLACE with this. The old code lives in DashboardPage now.

```
// src/App.tsx -- REPLACE the whole file
import { Routes, Route } from "react-router";
import DashboardPage from "../pages/DashboardPage";

function App() {
  return (
    <Routes>
      <Route path="/" element={<DashboardPage />} />
    </Routes>
  );
}

export default App;

// You should now see: your dashboard again, at http://localhost:5173/
// -- but now it is there because the URL '/' matched a Route, not
// because it was the only thing App could render.
//
// try it: http://localhost:5173/anything-else
// You get a BLANK page. No route matches, so the router renders
// nothing at all. We fix that with a catch-all route later today.
```

App.tsx Becomes a Route Table

Code Explanation

- **<Routes>** -- This tells React Router: "here is my whole list of URL rules. Look at the current URL and render *exactly one* of the children below."
- **<Route path="/" element={...} />** -- one rule. **path** is the URL to match, **element** is the JSX to render when it matches. Note it is **element={<Page />}**, *not* **component={Page}**.
- **Only one Route wins.** The router picks the single best match -- it does not render every route whose path happens to look similar.
- **No match means no output.** A URL nothing matches renders an empty page, not an error. That is why a catch-all **path="*" route** matters, and we add one later today.

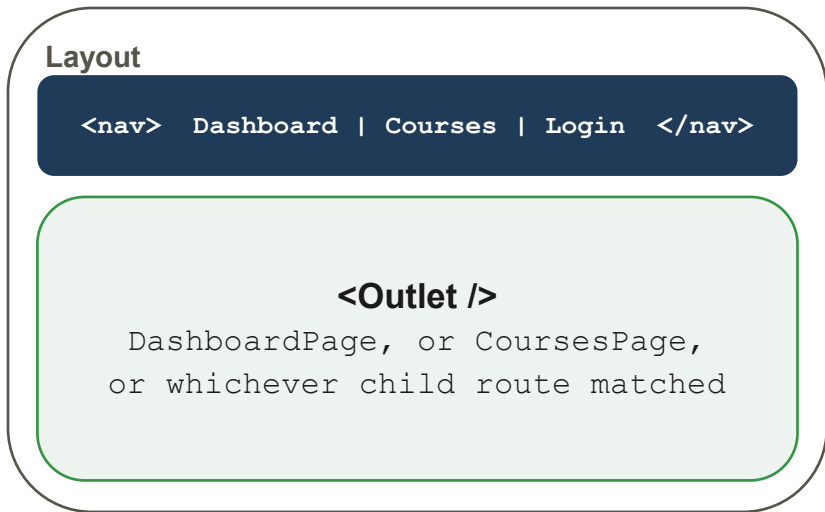
```
// src/App.tsx
<Routes>
  <Route path="/"
    element={<DashboardPage />} />
</Routes>

// element takes JSX: <Page />
// NOT a bare name:   Page
```

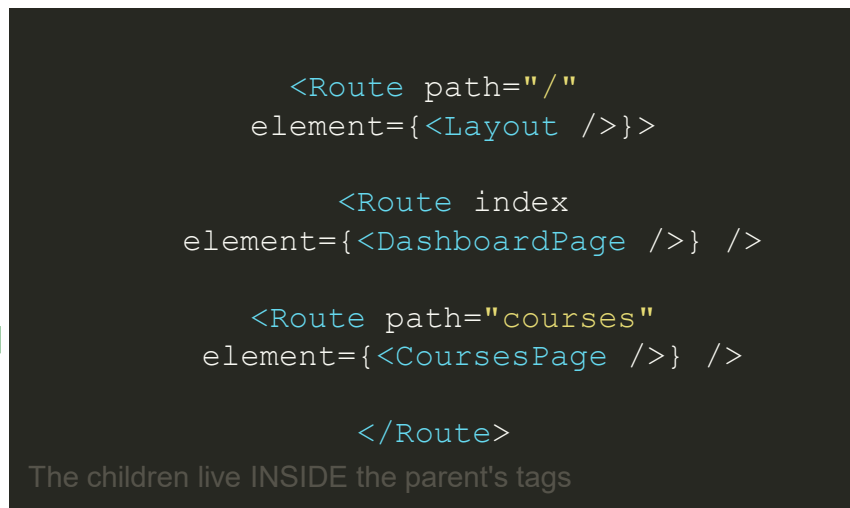
Nested Routes and Outlet

A parent route renders the frame; `<Outlet />` is the hole the child route fills

What the user sees



What you write in App.tsx



If the parent's element does not render `<Outlet />`, the child route still matches -- and nothing appears on screen. No error, no warning.

One Layout, Every Page

In the following slides, we will add:

- A small typed **Zustand** store in **src/store/authStore.ts** -- the nav bar has to know who is logged in
- A new **src/components/Layout.tsx** holding the nav bar, the dark mode toggle and an **<Outlet />**
- **NavLink** for the nav items, which styles itself differently when *its own* route is the active one
- The dark mode state moves out of the old App.tsx and into **Layout**, so every page inherits it
- **App.tsx** nests its routes *inside* the Layout route, and gains an **index** route

Files: **src/store/authStore.ts**, **src/components/Layout.tsx**, then **src/App.tsx**

What Is Zustand?

Before we write the store -- what a state library is for, and why useState is not enough

- **A place to keep state that more than one component needs.** Session 4's **useState** keeps a value inside *one* component. Who is logged in is needed by the nav bar, the guard, *and* the login page.
- **It removes prop drilling.** Without a store, the token has to be handed from **App** to **Layout** to every page, just so one button can read it -- and every file in between carries a value it never uses.
- **A store is a box that sits outside your components.** Any component reads from it directly with a hook. Nothing is passed down, so nothing in the middle has to know about it.
- **Zustand is the smallest of these libraries.** One function, **create()**, and your store *is* a hook. There is no Provider to wrap the app in, which is what makes it shorter than Redux or Context.

```
// WITHOUT a store: prop drilling
<Layout token={token}>
  <CoursesPage token={token}>
    <CourseCard token={token} />
  </CoursesPage>
</Layout>

// Layout and CoursesPage never use
// token. They only pass it along.

// WITH a store: read it directly
const token = useAuthStore(
  (state) => state.token
);
```

A Typed Zustand Store

A NEW file: src/store/authStore.ts. Create the src/store/ folder first.

```
// src/store/authStore.ts -- NEW FILE
// The nav bar we build next has to know whether anyone is logged in. A
// store is a box any component can read from, without passing props.
import { create } from "zustand";

// The shape of the store: its data AND the functions that change it
interface AuthState {
  token: string | null;
  userName: string | null;
  login: (name: string) => void;
  logout: () => void;
}

const useAuthStore = create<AuthState>((set) => ({
  token: null,
  userName: null,
  login: (name) => set({ token: `demo-token-${name}`, userName: name }),
  logout: () => set({ token: null, userName: null }),
}));

export default useAuthStore;

// You should now see: nothing at all change on screen. No component reads
// this store yet -- Layout starts using it two slides from now.
```

A Typed Zustand Store

Code Explanation

- **interface AuthState** -- describes the whole store: two values *and* two functions. **login: (name: string) => void** is a function type, the same syntax as Session 1's **Formatter** alias.
- **create<AuthState>((set) => ({ ... }))** -- This tells Zustand: "build me a store shaped like **AuthState**, and here is its starting state plus the actions that change it."
- **set({ ... })** -- the only way to change the store. Pass just the keys you want to change; Zustand merges them in and re-renders every component reading them.
- **useAuthStore((state) => state.userName)** -- a *selector*. It picks out one field, so the component re-renders when *that* field changes, not when some unrelated part of the store does.

```
// src/store/authStore.ts
const useAuthStore = create<AuthState>(
  (set) => ({ token: null, ... })
);

// how Layout reads it, next slide
const userName = useAuthStore(
  (state) => state.userName
);
```

The Layout Component (1 of 3)

A NEW file: src/components/Layout.tsx. Imports, state, and the link-styling helper.

```
// src/components/Layout.tsx -- NEW FILE
import { NavLink, Outlet } from "react-router";
import useToggle from "../hooks/useToggle";
import useAuthStore from "../store/authStore"; // built two slides ago

function Layout() {
  // Dark mode MOVES here, out of Session 5's App.tsx
  const [isDarkMode, toggleDarkMode] = useToggle(false);
  const userName = useAuthStore((state) => state.userName);
  const logout = useAuthStore((state) => state.logout);

  // The classes every nav link shares, then the two variants
  const base = "rounded px-3 py-1.5 text-sm";
  const activeLink = `${base} bg-blue-600 font-semibold text-white`;
  const idleLink = `${base} text-gray-700 hover:bg-gray-200 dark:text-gray-300`;

  // NavLink hands this function an isActive flag on every render
  const linkClass = ({ isActive }: { isActive: boolean }): string =>
    isActive ? activeLink : idleLink;

  // ... the JSX continues on the next slide -- do not type this line ...
}
```

The Layout Component (2 of 3)

src/components/Layout.tsx, continued -- the nav bar itself.

```
// src/components/Layout.tsx -- continues from the previous slide
return (
  <div className={isDarkMode ? "dark" : ""}>
    <div className="min-h-screen bg-gray-50 dark:bg-gray-900">

      <nav className="flex flex-wrap items-center gap-2 border-b
        border-gray-200 bg-white p-4 dark:border-gray-700
        dark:bg-gray-800">

        <span className="mr-4 font-bold text-gray-900 dark:text-white">
          Submission Tracker
        </span>

        <NavLink to="/" end className={linkClass}>Dashboard</NavLink>
        <NavLink to="/courses" className={linkClass}>Courses</NavLink>
        <NavLink to="/submissions" className={linkClass}>
          Submissions
        </NavLink>

      </nav>

    </div>
  </div>
);

// ... the nav bar continues on the next slide -- do not type this line ...
```

The Layout Component (3 of 3)

src/components/Layout.tsx, the last part -- the Logout button, the theme toggle, and the Outlet.

```
// src/components/Layout.tsx -- continues from the previous slide
{userName === null ? (
  <NavLink to="/login" className={linkClass}>Login</NavLink>
) : (
  <button onClick={logout}
    className="rounded px-3 py-1.5 text-sm text-gray-700
    dark:text-gray-300">
    Logout ({userName})
  </button>
)}

<button onClick={toggleDarkMode}
  className="ml-auto rounded bg-gray-800 px-3 py-1.5 text-sm
  text-white dark:bg-gray-200 dark:text-gray-900">
  {isDarkMode ? "Light Mode" : "Dark Mode"}
</button>
</nav>

<main className="p-6">
  <Outlet />          {/* <-- THE HOLE */>
</main>

</div>
</div>
);
}

export default Layout;
```

Nesting the Routes Inside Layout

In src/App.tsx: REPLACE the whole file again. The Route for '/' now wraps the others.

```
// src/App.tsx -- REPLACE the whole file
import { Routes, Route } from "react-router";
import Layout from "../components/Layout";
import DashboardPage from "../pages/DashboardPage";

function App() {
  return (
    <Routes>
      <Route path="/" element={<Layout />}>
        <Route index element={<DashboardPage />} />
      </Route>
    </Routes>
  );
}

export default App;

// Note what changed: the Route for '/' no longer SELF-CLOSES. It now
// has an opening tag, a child inside it, and a closing </Route>.
//
// You should now see: a white nav bar across the top with Dashboard,
// Courses, Submissions and Login in it, and your dashboard below it.
// The Dashboard link is highlighted blue because you are on '/'.
// try it: http://localhost:5173/
```


Nesting the Routes Inside Layout

Code Explanation

- **<Outlet />** -- This tells React Router: "whichever child route matched, render it *right here*." Without it the child matches but nothing appears on screen.
- **<Route index element={...} />** -- the child that shows at the parent's *own* URL. **index** is what you write instead of **path=""**, and it means 'the default child'.
- **NavLink vs Link** -- both navigate. **NavLink** additionally knows whether *it* is the current route, and hands **isActive** to a **className** function so it can style itself.
- **The end prop** -- **<NavLink to="/" end>** matches *only /*. Without **end**, **/** counts as a prefix of every URL and the Dashboard link would look active on every page.

```
// src/components/Layout.tsx
<main>
  <Outlet />
</main>

// src/App.tsx
<Route path="/" element={<Layout />}>
  <Route index element={<Dash />} />
</Route>
```

URL Parameters

In the following slides, we will add:

- **src/pages/CoursesPage.tsx** -- Session 5's course grid, moved, with each card wrapped in a **<Link>**
- **src/pages/CourseDetailPage.tsx** -- a page that reads the course code *out of the URL itself*
- A route with a **:code** segment -- **/courses/:code** -- and a typed **useParams<{ code: string }>()**
- **useNavigate()** for a Back button, which is how you navigate from inside an event handler

Files: **src/pages/CoursesPage.tsx**, **src/pages/CourseDetailPage.tsx**, **src/App.tsx**

The Courses Page (1 of 3)

A NEW file: src/pages/CoursesPage.tsx. This is Session 5's search + grid, moved.

```
// src/pages/CoursesPage.tsx -- NEW FILE
import { useState, useEffect, useRef } from "react";
import { Link } from "react-router"; // <-- NEW
import type { Course } from "../types/index";
import CourseCard from "../components/CourseCard";
import usePrevious from "../hooks/usePrevious";
import { allCourses } from "../data/mockData"; // <-- NEW

function CoursesPage() {
  // All six of these moved from Session 5's App.tsx, unchanged
  const [courses, setCourses] = useState<Course[]>([]);
  const [isLoading, setIsLoading] = useState<boolean>(true);
  const [isError, setIsError] = useState<boolean>(false);
  const [searchTerm, setSearchTerm] = useState<string>("");
  const searchInputRef = useRef<HTMLInputElement>(null);
  const previousSearch = usePrevious(searchTerm);

  useEffect(() => {
    setTimeout(() => {
      setCourses(allCourses); // <-- was setCourses([course])
      setIsLoading(false);
    }, 500);
  }, []);

  // ... continued on the next slide -- do not type this line ...
}
```

The Courses Page (2 of 3)



src/pages/CoursesPage.tsx, continued -- the handler, the filter, and the two early returns.

```
// src/pages/CoursesPage.tsx -- continues from the previous slide
const handleSearchChange = (
  e: React.ChangeEvent<HTMLInputElement>
): void => setSearchTerm(e.target.value);

// Matches the CODE as well as the title -- typing ITELECT has to work
const filteredCourses = courses.filter((c) =>
  c.title.toLowerCase().includes(searchTerm.toLowerCase()) ||
  c.code.toLowerCase().includes(searchTerm.toLowerCase())
);

if (isLoading) {
  return <div className="animate-pulse p-6">Loading courses...</div>;
}

if (isError) {
  return (
    <div className="rounded-lg bg-red-50 p-4 text-red-700">
      Could not load courses.
    </div>
  );
}

// ... the JSX continues on the next slide -- do not type this line ...
```

The Courses Page (3 of 3)

src/pages/CoursesPage.tsx, the JSX. The one genuinely new part is the `<Link>` around each card.

```
// src/pages/CoursesPage.tsx -- continues from the previous slide

return (
  <div>
    <h2 className="mb-4 text-2xl font-bold text-gray-900
      dark:text-white">Courses</h2>
    <button onClick={() => setIsError(true)}
      className="mb-2 rounded bg-red-100 px-2 py-1 text-xs text-red-700">
      Simulate Error
    </button>
    <input ref={searchInputRef} value={searchTerm}
      onChange={handleSearchChange} placeholder="Search courses..."
      className="w-full rounded border border-gray-300 p-2" />
    {previousSearch !== undefined && previousSearch !== searchTerm && (
      <p className="mt-1 text-sm text-gray-500">
        Previous search: "{previousSearch}"</p>
      )}
    <div className="mt-6 grid grid-cols-1 gap-4 sm:grid-cols-2
      lg:grid-cols-3">
      {filteredCourses.map((c) => (
        <Link key={c.code} to={`/${c.code}`}> { /* <-- NEW */}
        <CourseCard course={c} />
      </Link>
      ))}
    </div>
  </div>
);
}
export default CoursesPage;
```

Reading a Value Out of the URL (1 of 2)



A NEW file: `src/pages/CourseDetailPage.tsx`. Imports, the two hooks, and the lookup.

```
// src/pages/CourseDetailPage.tsx -- NEW FILE
import { useParams, useNavigate } from "react-router";
import CourseCard from "../components/CourseCard";
import { allCourses } from "../data/mockData";

function CourseDetailPage() {
  // Reads whatever is in the :code slot of the URL
  const { code } = useParams<{ code: string }>();
  const navigate = useNavigate();

  // Turn that string into a real Course object
  const course = allCourses.find((c) => c.code === code);

  // The URL is user input -- they can type anything. Handle that.
  if (course === undefined) {
    return (
      <div className="rounded-lg bg-red-50 p-4 text-red-700">
        No course found with code "{code}".
      </div>
    );
  }

  // ... the JSX continues on the next slide -- do not type this line ...
}
```

Reading a Value Out of the URL (2 of 2)



src/pages/CourseDetailPage.tsx, the JSX. This closes the file.

```
// src/pages/CourseDetailPage.tsx -- continues from the previous slide
return (
  <div>
    <h2 className="mb-4 text-2xl font-bold text-gray-900
      dark:text-white">{course.title}</h2>

    <div className="max-w-sm">
      <CourseCard course={course} />
    </div>

    <button onClick={() => navigate("/courses")}
      className="mt-4 rounded bg-blue-600 px-3 py-1.5 text-sm
        font-semibold text-white transition hover:bg-blue-700">
      Back to Courses
    </button>
  </div>
);
}

export default CourseDetailPage;
```

Adding the Two Course Routes

In src/App.tsx: add two imports and two child routes. Everything else in the file stays.

```
// src/App.tsx
import { Routes, Route } from "react-router"; // already there
import Layout from "../components/Layout"; // already there
import DashboardPage from "../pages/DashboardPage"; // already there
import CoursesPage from "../pages/CoursesPage"; // <-- NEW
import CourseDetailPage from "../pages/CourseDetailPage"; // <-- NEW

function App() {
  return (
    <Routes>
      <Route path="/" element={<Layout />} />
      <Route index element={<DashboardPage />} />
      <Route path="courses" element={<CoursesPage />} />
      <Route path="courses/:code" element={<CourseDetailPage />} />
    </Route>
  </Routes>
);
}

export default App;

// The index route was already there. The two 'courses' routes are NEW.
// Inside JSX you must use {/* ... */}, never // -- see slide 9.

// try it: http://localhost:5173/courses
// try it: http://localhost:5173/courses/CSSWENG <-- type this by hand
// try it: http://localhost:5173/courses/BANANA <-- the red message
```


Reading a Value Out of the URL

Code Explanation

- **path="courses/:code"** -- the colon marks a *placeholder*. This one Route matches **/courses/ITELECT4**, **/courses/CSSWENG**, and any other value in that position.
- **useParams<{ code: string }>()** -- This tells React Router: "give me the placeholder values out of the URL -- and TypeScript, they look like this." The name **code** must match the **:code** in the path exactly.
- **The value is always string | undefined**, even though we typed it. TypeScript cannot promise the URL contains anything, so the **course === undefined** check is required, not optional politeness.
- **useNavigate()** returns a *function* you call later, from inside an event handler -- **navigate("/courses")**. Use **<Link>** for something the user clicks directly.

```
// src/App.tsx
<Route path="courses/:code"
  element={<CourseDetailPage />} />

// src/pages/CourseDetailPage.tsx
const { code } =
  useParams<{ code: string }>();

// code is string | undefined
```

Link vs useNavigate vs Navigate

Three ways to change the URL. Picking the wrong one is the most common routing mistake.

- **<Link to="/courses">** -- for something the user *clicks*. It renders a real **<a>** tag, so middle-click and 'open in new tab' work. Reach for this first.
- **useNavigate()** -- for navigating *after something happens*: a form submitted, a login succeeded, a Back button pressed. It gives you a function to call inside a handler.
- **<Navigate to="/login" replace />** -- a *component* you render instead of a page, to redirect immediately. This is what **ProtectedRoute** uses in the next section.
- **The trap:** never call **navigate(...)** in the body of a component. It runs on every render and loops forever. Inside an event handler, or as **<Navigate />**, is safe.

```
// the user clicks it
<Link to="/courses">Courses</Link>

// after something happens
const navigate = useNavigate();
onClick={ () => navigate("/courses") }

// redirect instead of rendering
return <Navigate to="/login" replace />;
```

Logging In + Protecting a Route

In the following slides, we will add:

- A new **src/pages/LoginPage.tsx** that calls the **login()** action we wrote earlier, then **navigate()**s away
- A new **src/components/ProtectedRoute.tsx** that renders **<Outlet />** or redirects to **/login**
- **SubmissionBadge** finally gets its Tailwind classes, because it is about to appear on screen
- **SubmissionsPage.tsx**, **NotFoundPage.tsx**, and the finished route table

Files: **LoginPage.tsx**, **ProtectedRoute.tsx**, **SubmissionBadge.tsx**, **SubmissionsPage.tsx**, **App.tsx**

The Login Page (1 of 2)

A NEW file: src/pages/LoginPage.tsx. There is no real password -- typing a name is enough.

```
// src/pages/LoginPage.tsx -- NEW FILE
import { useState } from "react";
import { useNavigate } from "react-router";
import useAuthStore from "../store/authStore";

function LoginPage() {
  const [name, setName] = useState<string>("");

  // Pull just the login action out of the store
  const login = useAuthStore((state) => state.login);
  const navigate = useNavigate();

  const handleLogin = (): void => {
    login(name); // 1. put the token in the store
    navigate("/submissions"); // 2. then send them where they were going
  };

  // ... the JSX continues on the next slide -- do not type this line ...
}
```

The Login Page (2 of 2)

src/pages/LoginPage.tsx, the JSX -- plus the route that renders it.

```
// src/pages/LoginPage.tsx -- continues from the previous slide
return (
  <div className="max-w-sm">
    <h2 className="mb-4 text-2xl font-bold text-gray-900
      dark:text-white">Login</h2>

    <input value={name} onChange={(e) => setName(e.target.value)}
      placeholder="Your name"
      className="w-full rounded border border-gray-300 p-2" />

    <button onClick={handleLogin} disabled={name === ""}
      className="mt-3 rounded bg-blue-600 px-3 py-1.5 text-sm
        font-semibold text-white transition hover:bg-blue-700
        disabled:bg-gray-400">
      Log In
    </button>
  </div>
);
}

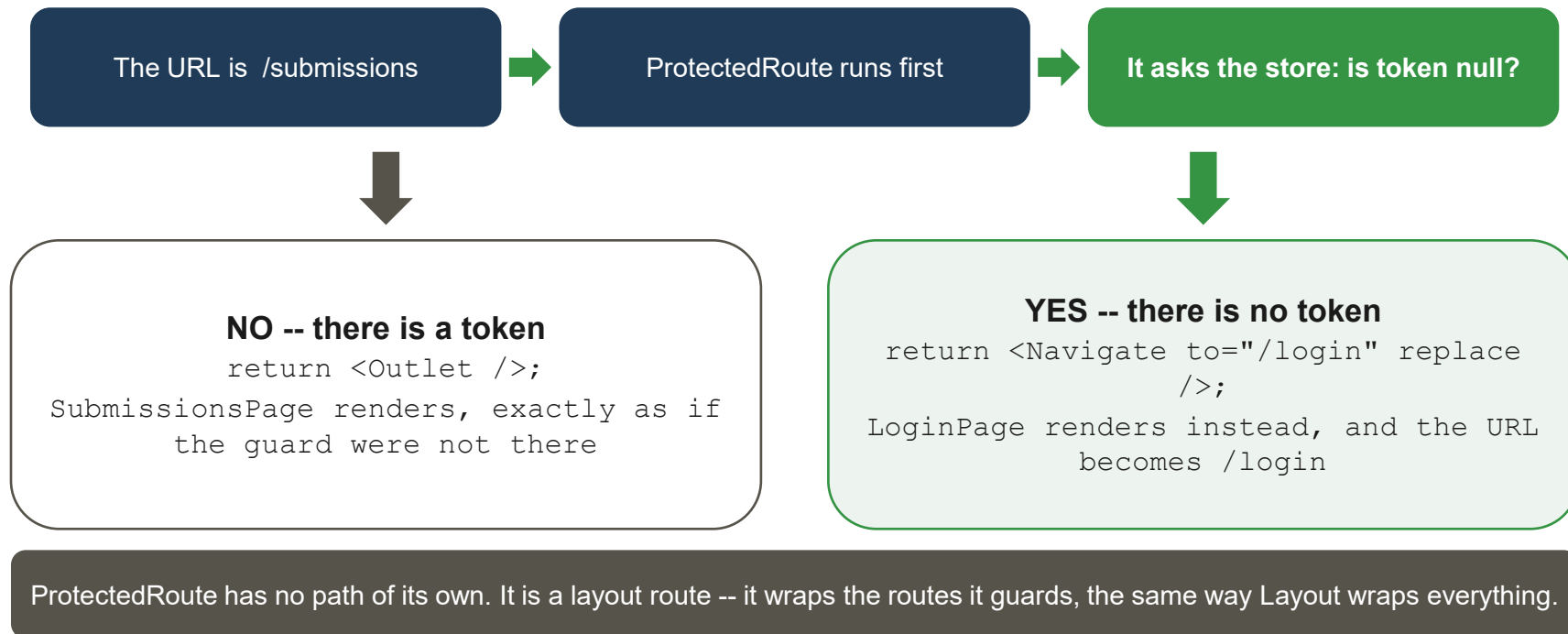
export default LoginPage;

// ----- Also in src/App.tsx, with the other child routes -----
<Route path="login" element={<LoginPage />} />

// You should now see: log in with any name, and the nav bar swaps
// from Login to Logout (your name). That alone proves the store works.
// /submissions is still blank -- we build it in four slides.
```

How ProtectedRoute Decides

One component, one question, two possible answers



The Protected Route

A NEW file: `src/components/ProtectedRoute.tsx`. It renders no UI of its own -- it only decides.

```
// src/components/ProtectedRoute.tsx -- NEW FILE
import { Navigate, Outlet } from "react-router";
import useAuthStore from "../store/authStore";

function ProtectedRoute() {
  const token = useAuthStore((state) => state.token);

  // No token? Send them to the login page instead of the page
  // they asked for.
  if (token === null) {
    return <Navigate to="/login" replace />;
  }

  // There IS a token, so render whichever child route matched.
  return <Outlet />;
}

export default ProtectedRoute;

// Notice: this component has NO path of its own. It is a layout route,
// exactly like Layout -- it wraps the routes it guards.
```

Styling SubmissionBadge

In src/components/SubmissionBadge.tsx -- only the className values change.

```
// src/components/SubmissionBadge.tsx
// GT2 asked you to style all THREE components. Our class demo only ever
// styled two, because SubmissionBadge had nowhere to appear. It appears
// on the next slide, so it gets its classes now.

// ... imports and the SubmissionBadgeProps interface: ALL UNCHANGED ...

return (
  // <-- only this block changes
  <div className="rounded-lg border border-gray-200 bg-white p-5
    shadow-sm dark:border-gray-700 dark:bg-gray-800">
    <p className="text-gray-900 dark:text-white">
      Repo: {submission.repoUrl}
    </p>
    <p className="text-sm text-gray-500 dark:text-gray-400">
      Score: {submission.score ?? "Not graded yet"}
    </p>
    {children}
  </div>
);

// ... export default SubmissionBadge; UNCHANGED ...
```


The Submissions Page

A NEW file: `src/pages/SubmissionsPage.tsx` -- `SubmissionBadge` finally reaches the screen.

```
// src/pages/SubmissionsPage.tsx -- NEW FILE
import SubmissionBadge from "../components/SubmissionBadge";
import { allSubmissions } from "../data/mockData";

function SubmissionsPage() {
  return (
    <div>
      <h2 className="mb-4 text-2xl font-bold text-gray-900 dark:text-white">My Submissions</h2>

      <div className="grid grid-cols-1 gap-4 sm:grid-cols-2">
        {allSubmissions.map((s) => (
          <SubmissionBadge key={s.id} submission={s}>
            <p className="text-sm text-gray-500 dark:text-gray-400">
              Course: {s.courseCode}</p>
            </SubmissionBadge>
          </>
        ))}
      </div>
    </div>
  );
}

export default SubmissionsPage;

// The <p> is passed as CHILDREN to SubmissionBadge -- the typed-children
// pattern from Session 3, finally used for real.
```

The 404 Page

A NEW file: `src/pages/NotFoundPage.tsx`. This is the fix for the blank page from slide 14.

```
// src/pages/NotFoundPage.tsx -- NEW FILE
import { Link } from "react-router";

function NotFoundPage() {
  return (
    <div>
      <h2 className="mb-2 text-2xl font-bold text-gray-900 dark:text-white">
        404 -- Page Not Found
      </h2>
      <Link to="/" className="text-blue-600 underline hover:text-blue-700">
        Go back to the Dashboard
      </Link>
    </div>
  );
}

export default NotFoundPage;

// Remember slide 14, where /anything-else gave a blank white page?
// The route that renders THIS page is on the next slide, and it is
// the last thing we add today.
```

The Finished Route Table

In src/App.tsx: REPLACE the whole file one last time. This is the complete, final file.

```
// src/App.tsx -- REPLACE the whole file (final version)
import { Routes, Route } from "react-router";
import Layout from "../components/Layout";
import ProtectedRoute from "../components/ProtectedRoute";
import DashboardPage from "../pages/DashboardPage";
import CoursesPage from "../pages/CoursesPage";
import CourseDetailPage from "../pages/CourseDetailPage";
import LoginPage from "../pages/LoginPage";
import SubmissionsPage from "../pages/SubmissionsPage";
import NotFoundPage from "../pages/NotFoundPage";

function App() {
  return (
    <Routes>
      <Route path="/" element={<Layout />} />
      <Route index element={<DashboardPage />} />
      <Route path="courses" element={<CoursesPage />} />
      <Route path="courses/:code" element={<CourseDetailPage />} />
      <Route path="login" element={<LoginPage />} />
      <Route element={<ProtectedRoute />} /* <-- the guard */
        <Route path="submissions" element={<SubmissionsPage />} />
      </Route>
      <Route path="*" element={<NotFoundPage />} />
    </Route>
  </Routes>
);
}

export default App;
```

The Finished Route Table

Code Explanation

- **<Route element={<ProtectedRoute />}>** with *no path* -- a pathless layout route. It adds a guard to everything nested inside it without adding anything to the URL.
- **<Navigate to="/login" replace />** -- **replace** overwrites the current history entry instead of adding one. Without it, Back from **/login** returns to **/submissions**, which redirects to **/login** again -- so Back appears frozen.
- **path="*" --** the catch-all. It matches any URL no other route claimed, and it is the fix for the blank page we saw on slide 14.
- **Order does not decide the winner.** React Router scores routes by specificity, so "*" loses to every real path no matter where you write it. Writing it last is a readability convention, not a rule.

```
// src/App.tsx
<Route element={<ProtectedRoute />}>
  <Route path="submissions"
    element={<SubmissionsPage />} />
</Route>

<Route path="*"
  element={<NotFoundPage />} />
```

Your New File Tree

Nine new files, three new folders -- and App.tsx is far shorter than it was previously

- **src/pages/** -- one file per screen. Every one is a plain component; nothing about them is 'routery' except being named in the route table.
- **src/components/** -- gains **Layout.tsx** and **ProtectedRoute.tsx**, which are components that render *structure* rather than content.
- **src/store/** and **src/data/** -- new homes for state and mock data that more than one page needs. Both grow in Session 7.
- **src/App.tsx** -- now the only place in the codebase that knows the URL map. To find out what pages exist, you read *one* file.

```
src/
  App.tsx           (route table)
  main.tsx          (BrowserRouter)
  components/
    CourseCard.tsx
    Layout.tsx      NEW
    ProtectedRoute.tsx NEW
    SubmissionBadge.tsx
    UserCard.tsx
  data/              NEW
    mockData.ts
  hooks/
    usePrevious.ts
    useToggle.ts
  pages/             NEW
    CourseDetailPage.tsx
    CoursesPage.tsx
    DashboardPage.tsx
    LoginPage.tsx
    NotFoundPage.tsx
    SubmissionsPage.tsx
  store/             NEW
    authStore.ts
  types/
    index.ts
```

What You Should See

Run `npm run dev` and click through all eight. This checks the CLASS DEMO -- GT3 comes next.

```
// 1. The nav bar is on every page, and never reloads when you click it.
//   try it: http://localhost:5173/

// 2. Dashboard is highlighted blue on '/', Courses is highlighted on
//   '/courses'. Only ever one at a time.

// 3. Three course cards appear, after a half-second pulse.
//   try it: http://localhost:5173/courses

// 4. Typing ITELECT in the search box leaves two cards; typing SOFT
//   leaves one. The filter matches the title AND the code.

// 5. Clicking a card opens its detail page, and the URL says which one.
//   try it: http://localhost:5173/courses/CSSWENG

// 6. A code that does not exist shows the red message, not a crash.
//   try it: http://localhost:5173/courses/BANANA

// 7. Submissions bounces you to Login while logged out. Log in with any
//   name and you land on Submissions; the nav swaps to Logout (name).
//   try it: http://localhost:5173/submissions

// 8. A nonsense URL shows the 404 page, WITH the nav bar still on it.
//   try it: http://localhost:5173/banana

// Dark mode still works on every page. Back and Forward both still work.
```

Traps to Remember

Six ways this goes wrong. Five of them produce no error message at all.

- **Importing from react-router-dom.** The package does not exist in v8. Symptom: *Failed to resolve import*. Fix: drop the **-dom**.
- **Forgetting <Outlet /> in a layout.** The child route matches and renders *nothing*. Symptom: the nav bar appears, the page below it is empty. No error.
- **A :param name that does not match useParams.** `path="courses/:code"` with `useParams<{ id: string }>()` gives you **undefined**, silently. The two words must be identical.
- **Using // inside JSX.** It is not a comment there -- React prints it on the page. Use `{/* ... */}`.
- **Calling navigate() in the component body.** Infinite loop. Symptom: *Maximum update depth exceeded*, tab freezes. It belongs in an event handler.
- **Redirecting without replace.** The Back button appears broken, because you bounce straight forward again.

```
// silent: nav shows, page empty
function Layout() {
  return <nav>...</nav>;
} // <-- no <Outlet />
```

```
// silent: code is undefined
<Route path="courses/:code" ...>
useParams<{ id: string }>()
```

```
// loud: the tab freezes
function Page() {
  navigate("/"); // every render
}
```

GT3 - Part 1 of 3

Graded Task | Individual | Pushed Today -- GT3 is Submitted at the End of Session 8

What to do:

- Install react-router and zustand in your own GT2 project
- Wrap `<App />` in `<BrowserRouter>` inside `src/main.tsx`
- Create `src/pages/` with at least 4 page components of your own
- Create `src/components/Layout.tsx` with a shared nav and an `<Outlet />`
- Move your GT2 UI into those pages -- `App.tsx` must contain only the route table
- Add one route with a URL parameter, read with a typed `useParams<{ ... }>()`
- Use `useNavigate()` at least once, from inside an event handler
- Create `src/store/authStore.ts` -- a Zustand store typed with an interface, holding token plus login and logout
- Create `ProtectedRoute.tsx`: `<Navigate to="/login" replace />` when token is null, `<Outlet />` otherwise
- Add a `path=""` route so that no URL gives a blank page
- `npm run build` must finish with ZERO TypeScript errors

Reminders

- Your routes must describe YOUR app's entities, not `/courses`. A lost-and-found app routes `/items/:id`; an RSVP app routes `/events/:id`.
- Do not copy the class demo. Its pages exist to show the pattern -- the graded work is applying that pattern to the app you chose in Session 2.
- NEW THIS MODULE: work on a branch named `gt3-part1` and merge it with a pull request. Committing straight to main costs marks under Commit history and process.
- Push only today -- do NOT tag. GT3 is tagged and submitted at the end of Session 8.
- Be ready to explain any line if you are spot-checked. That is 10 of the 100 points.
- Session 7 adds TanStack Query on top of these routes. A broken route table today blocks next week.

Branch, Commit, Pull Request

New from this module onward: every GT part is a feature branch merged through a pull request.

Before you open the pull request, check:

- **npm run build** finishes with zero TypeScript errors -- not just **npm run dev**, which skips type checking
- All eight items on the *What You Should See* slide actually work in your own app
- **src/App.tsx** contains *only* the route table -- no leftover UI from GT2
- You are on the **gt3-part1** branch, *not* on **main**, before you commit

Terminal

```
> git checkout -b gt3-part1
> npm run build
> git add .
> git commit -m "GT3 Part 1: React Router, Layout, protected route"
> git push -u origin gt3-part1

// Then, on github.com:
//   Compare & pull request -> base: main <- compare: gt3-part1
//   Title it: GT3 Part 1 - Routing and Navigation
//   Merge it yourself. Leave the branch in place, do not delete it.
//
// Do NOT run git tag today. GT3 is tagged at the end of Session 8.
```

Before Next Session

Session 7 | Aug 15, 2026 | Module 3 | State Management + Data Fetching

For next session, you need to:

- Push your **gt3-part1** branch and merge the pull request -- Session 7 builds directly on these routes
- Read the **Zustand** TypeScript guide and skim the **TanStack Query** quickstart
- Confirm **npm run build** finishes with zero errors -- a broken build blocks next week's work
- Re-read today's **Traps to Remember** slide if any of your routes still misbehave

Next session topics:

- Building the Zustand store out properly -- typed actions and selectors
- **TanStack Query** -- typed **useQuery** and **useMutation**
- Replacing the fake **setTimeout** fetch with real API data
- **GT3 Part 2** -- live data rendering inside today's routes



ITELECT4 | Session 6 End

1962 J.P Laurel Highway Lipa City 4217
Batangas, Philippines

www.dlsl.edu.ph